Group ID - 34
Automatic Cricket Highlight Generation via Multimodal Fusion of Scoreboard OCR, Audio Excitement Cues, and LLM-based Commentary Understanding

Kuldip Manvar (240575) Yash Pandit (231183) Aryan Kadam (230221)
Yash Kumar (231186) Sanjay Jangir (230913)

Abstract

Cricket matches span 3–8 hours, making manual highlight curation prohibitively labour-intensive. We present a modular, interpretable pipeline for automatic cricket highlight generation that fuses three complementary modalities via a layered sensor–brain–scalpel *architecture: (i) a* ***Vision Sensor*** *comprising a YOLOv26-based scoreboard detector coupled with EasyOCR for parsing the on-screen score, with Otsu-binarised preprocessing, a persistence filter that rejects transient OCR errors, and a rule-based validator that classifies run/wicket deltas into* `Four`*,* `Six`*, and* `Wicket` *events; (ii) an* ***Audio Sensor*** *that computes a composite robust z-score over loudness, spectral flux, delta-loudness, spectral centroid, and crest factor to detect crowd-roar peaks and sustained excitement regions; and (iii) a* ***Brain*** *layer in which a Whisper-transcribed commentary track and the OCR events are jointly processed by a two-stage Gemini LLM — Flash for screening, Pro for verification — operating on* targeted Regions of Interest *(not the full transcript) to produce semantically-grounded, deduplicated highlight candidates. A final* ***Scalpel*** *layer anchors each verified event to the nearest audio peak and snaps clip boundaries to HSV-histogram-based scene changes before extraction via FFmpeg re-encoding. We additionally study three alternative approaches that informed our final design: a YouTube "most-replayed" SVG-heatmap scraper (solved analytically via cubic Bézier inversion), a dense optical-flow motion-intensity cue (Farnebäck) used as a fallback within the OCR pipeline, and a scoreboard-only baseline without any audio/LLM signal. On a full T20 match (ENG vs. IND), the fused pipeline identifies the large majority of boundaries and wickets while remaining fully interpretable, with no end-to-end training required.*

1. Introduction

The volume of long-form sports content uploaded to the web every day has grown faster than human editors can summarise it. A single men's T20 International runs roughly 3.5 hours; an ODI 8 hours; a Test match five days. Fans, broadcasters, and analysts all want the same thing: a short, faithful clip-reel containing the *actual* boundaries, wickets, and decisive moments, assembled from the raw broadcast with no manual editing.

The challenge is that a cricket "highlight" is not a single easily-detected visual pattern. The only frame truly shared by every wicket is the instant the batter walks off — which happens several seconds *after* the event. A six looks identical to a well-timed four in the first five frames. The crowd reacts differently to a home-team wicket than to an away-team six. No single cue is sufficient; every cue is noisy.

We therefore take a deliberately *multimodal and interpretable* stance. Rather than train a monolithic deep network (which would require tens of hours of frame-level labels that do not exist publicly), we build a layered pipeline in which each layer exploits a different signal that broadcasters already provide:

- The **scoreboard** is always on screen and its run/wicket delta is a ground-truth signal of what happened. An OCR module with a persistence filter gives us semantically-labelled events (Four, Six, Wicket) with timestamps, essentially for free.
- The **audio track** carries crowd and commentator reactions that almost always spike at decisive moments, even when the visual scene is unremarkable. A streaming spectral-feature analyser gives us a modality-independent confirmation signal.
- The **commentary transcript** (obtained via Whisper) gives us *semantic* context that neither pixels nor loudness can: it knows the difference between a review and a wicket, between a near-six and a six, and between a commentator-milestone remark ("half-century!") and

routine banter. An LLM processes only *targeted regions of interest* identified by the OCR and keyword triggers, making it both efficient and focused.

Rather than treating these three modalities as independent candidate streams that are naively pooled, we organise them into a **layered architecture** (Figure 1): two *Sensors* (Vision and Audio) feed a *Brain* (targeted LLM verification) whose verified events are then *anchored* by a *Scalpel* layer that uses audio peaks and scene-change boundaries to produce frame-accurate, audio-synchronised clips.

Contributions. Our contributions are:

1. A layered sensor–brain–scalpel pipeline (Figure 1) in which OCR and audio independently propose candidates, an LLM verifies them on targeted ROIs, and a final anchoring step snaps clip boundaries to audio peaks and scene changes. Each layer is interpretable, tunable, and runs on modest hardware.
2. A persistence-filtered scoreboard OCR module with Otsu-binarised preprocessing, upscaling of small crops, and a domain-rule validator that maps integer deltas in the `(runs, wickets)` state to {Four, Six, Wicket, New Innings} labels, rejecting implausible transitions without any learned component.
3. A streaming audio analyser that computes a *robust* composite z-score (median/MAD, not mean/σ) over five features — dB-RMS, peak-RMS crest factor, spectral flux, spectral centroid, and Δ-dB — and detects both instantaneous peaks and *sustained* loudness regions, which catch extended celebrations that a peak-only detector misses.
4. A two-stage LLM commentary verifier: Gemini Flash cheaply screens each Region of Interest for highlight presence, and Gemini Pro performs detailed event extraction with Pydantic schema enforcement only on positively-screened regions. A replay-suppression deduplicator removes events of the same type within 45 s of each other.
5. An HSV-histogram scene-change detector that snaps clip start/end times to stable shot boundaries, combined with audio-peak anchoring and event-type-specific padding profiles (e.g. 75 s post-padding for DRS reviews, 22 s for wickets).
6. Three explored alternatives that we report honestly: a YouTube SVG-heatmap scraper (Bézier path inversion), a Farnebäck dense optical-flow motion cue used as a fallback within the OCR pipeline, and a scoreboard-only baseline. We explain why each was not adopted as the primary method.

2. Related Work

Sports highlight generation. Early approaches relied on hand-crafted shot-boundary detection and motion features [3]; broader video summarization surveys cover these well. Scoreboard-OCR has been explored for football and basketball [7] where the on-screen clock/score directly encodes events. Audio-based excitement detection via MFCC or RMS-energy models is standard [2]. Multimodal fusion combining audio, motion, and text was studied for soccer [1]. Automatic cricket highlight generation using event-driven and excitement-based features — replays, audio intensity, and player celebrations — has also been ex-

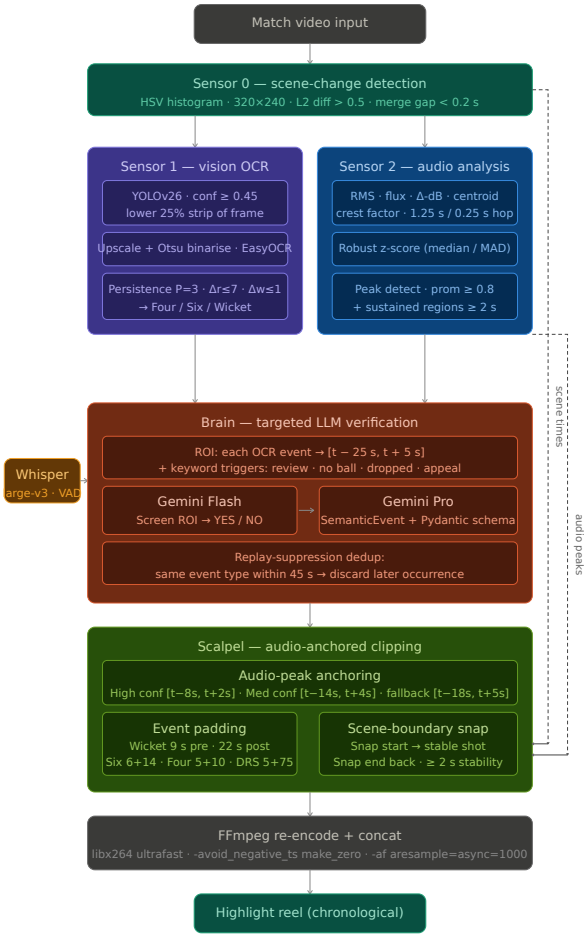

Figure 1. Overview of the proposed layered highlight generation pipeline. **Sensor 0:** HSV-histogram scene-change detection (one-time pass). **Sensor 1 (Vision):** YOLOv26 + EasyOCR + Otsu preprocessing + persistence filter + rule validator. **Sensor 2 (Audio):** streaming composite robust z-score with peak and sustained detectors. **Brain:** OCR events and keyword triggers define Regions of Interest; Gemini Flash screens each ROI, Gemini Pro extracts verified `SemanticEvent` objects; replay deduplication suppresses near-duplicates. **Scalpel:** each verified event is anchored to the nearest audio peak and clip boundaries are snapped to scene changes, with event-type-specific padding. Clips are extracted via FFmpeg re-encoding and concatenated chronologically.

plored [4]. End-to-end deep networks [5, 6] require large labelled datasets that are not publicly available for cricket.

Why we avoid end-to-end learning. Training a supervised highlight classifier on cricket frames needs frame-accurate event annotations across hundreds of hours — a dataset that does not exist in the public domain. Crowd-only audio classifiers fail on matches where the home crowd dominates, under-reacting to opposition wickets and over-reacting to even routine boundaries by the home side. Our design therefore treats the scoreboard — which is an *objective* signal placed there by the broadcaster — as the primary oracle, and uses audio and commentary as orthogonal verification and anchoring channels.

Novelty relative to the field. To our knowledge the combination of (a) a persistence-filtered rule-based scoreboard validator with Otsu-binarised OCR preprocessing, (b) a robust median/MAD composite audio detector that explicitly models *sustained* regions (not just peaks), (c) a two-stage LLM commentary verifier (Flash screening + Pro extraction) operating on targeted OCR-driven Regions of Interest rather than the full transcript, and (d) an HSV-histogram scene-change detector for clip-boundary snapping with event-type-specific padding, has not appeared together in the cricket-highlight literature. The pipeline is fully interpretable layer-by-layer and runs on a single Kaggle T4 GPU.

3. Proposed Method

Figure 1 shows the full pipeline. The input is a single match video V with duration T; the output is a set of highlight clips concatenated chronologically. We organise the pipeline into four layers: Sensor 0 (scene detection), Sensor 1 (vision/OCR), Sensor 2 (audio), the Brain (LLM verification), and the Scalpel (audio-anchored clipping).

3.1. Sensor 0: Scene-Change Detection

Before any modality-specific processing, we perform a single pass over the entire video with a lightweight **HSV-histogram scene-change detector**. For each frame (downscaled to 320×240), we compute a joint hue–saturation histogram (50×60 bins) and flag timestamps where the L2 distance between consecutive normalised histogram vectors exceeds a threshold of 0.5. Raw detections closer than 0.2 s are merged by averaging. The resulting list of scene-boundary timestamps is stored for use in the Scalpel layer (Sec. 3.5), where clip start/end times are snapped to stable shot boundaries.

3.2. Sensor 1: Scoreboard OCR Pipeline

Scoreboard localisation (YOLOv26). Modern cricket broadcasts composite a score overlay (the "bug") in a broadcaster-dependent region, typically the lower strip of the frame. We trained a `YOLOv26` detector on a custom Roboflow dataset of cricket frames with the scoreboard bug annotated across T20, ODI, and Test formats. For every sampled frame the YOLO head predicts bounding boxes with objectness and class confidence; the final per-box score is

$$s = P(\text{scoreboard}) \cdot \text{IoU}_{\text{pred}}^{\text{truth}}, \tag{1}$$

and we retain the highest-confidence box above a threshold of 0.45. To suppress spurious detections in the upper two-thirds of the frame (player close-ups, replays, graphics) we restrict YOLO inference to the bottom 25% strip of the image, which both speeds up inference and eliminates a large class of false positives.

Figure 2. Scoreboard detection examples from two different broadcasts. The YOLOv26 detector localises the score overlay (blue bounding box) with confidence scores of 0.91 (PAK vs. IND, top) and 0.98 (SA vs. NZ, bottom), demonstrating robustness across different broadcaster overlay styles.

OCR with preprocessing. The cropped scoreboard ROI undergoes enhanced preprocessing before OCR. Small crops (where the shorter side is below 500 px) are **upscaled** by a linear-interpolation resize so that the minimum dimension reaches 500 px. The upscaled crop is then converted to greyscale and binarised using **Otsu's method**:

$$I_{\text{bin}}(x, y) = \begin{cases} 255 & I(x, y) > t^{\star}_{\text{Otsu}} \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

where $t^{\star}_{\text{Otsu}}$ is the threshold that maximises inter-class variance. This Otsu-binarised image is passed to `EasyOCR`. A confusable-character substitution (`O`$\to$`0`, `l/I/i`$\to$`1`, `S/s`$\to$`5`, `B`$\to$`8`, `Z/z`$\to$`2`) is applied to the raw OCR text, non-alphanumeric characters except `/` and `-` are stripped, and the result is matched against the regex

```
(\d{1,3})\s*[/|-]\s*(\d{1,2})
```

which captures the canonical `runs/wickets` format.

Persistence filter. A single-frame OCR reading is unreliable: a transient motion blur or a replay graphic momentarily covering the score will produce junk numbers. We

therefore require that the *same* `(runs, wickets)` tuple be observed in $P = 3$ consecutive sampled frames (sample rate $= 1$ Hz) before accepting it as the current state. Formally, given a sliding buffer of the last P readings, we emit a stable score iff all P entries are identical.

Rule-based event validator. Let (r_t, w_t) denote the stable score at time t and (r_{t-1}, w_{t-1}) the previous stable score. We accept a transition iff

$$0 \leq \Delta r \leq 7 \quad \text{and} \quad 0 \leq \Delta w \leq 1, \tag{3}$$

(physically impossible deltas are silently rejected as OCR errors), and emit a labelled event according to:

$\Delta r = 4, \Delta w = 0$	`Four`
$\Delta r = 6, \Delta w = 0$	`Six`
$\Delta w = 1$	`Wicket`
$r_t = 0, w_t = 0$ after innings 1	`New Innings`

Single-run, double-run and triple-run deltas are recognised but not treated as highlight-worthy. New-innings events are tracked via an `innings_active` flag to avoid duplicate detections when the score remains at 0/0 for multiple frames.

3.3. Sensor 2: Audio Excitement Analysis

The audio track is extracted with `ffmpeg` as mono 16 kHz PCM. We then stream through the wave file using `soundfile` with a 1.25 s window and 0.25 s hop, computing five features per window:

- **RMS loudness** (dB): $\text{RMS}_{\text{dB}} = 20 \log_{10} \sqrt{\frac{1}{N} \sum_n x[n]^2}$
- **Peak loudness** (dB): $\text{Peak}_{\text{dB}} = 20 \log_{10} \max_n |x[n]|$
- **Crest factor** (dB): $\text{Crest}_{\text{dB}} = \text{Peak}_{\text{dB}} - \text{RMS}_{\text{dB}}$
- **Spectral flux** against the previous window's normalised magnitude spectrum m_{t-1}:

$$\phi_t = \sqrt{\frac{1}{K} \sum_k [\max(m_t[k] - m_{t-1}[k], 0)]^2} \tag{4}$$

- **Normalised spectral centroid**: $c_t = \frac{\sum_k f_k \, m_t[k]}{\sum_k m_t[k]} \cdot \frac{2}{f_s}$
- **Δ-loudness**: first difference of RMS_{dB} (captures sudden swells).

Robust composite score. Crowd audio is extremely heavy-tailed: a handful of celebrations dominate the variance, so mean/σ z-scoring pathologically under-weights them. We instead use the *robust* z-score based on the median and MAD:

$$z(x) = \frac{0.6745\,(x - \text{median}(x))}{\text{MAD}(x)}, \tag{5}$$

where $\text{MAD}(x) = \text{median}(|x - \text{median}(x)|)$ and the 0.6745 factor makes z a consistent estimator of the Gaussian standard score when the data are in fact Gaussian. The composite score is a weighted sum of the per-feature robust z-scores,

$$S_t = 0.55\, z_{\text{RMS}} + 0.25\, z_\phi + 0.10\, z_\Delta + 0.05\, z_c + 0.05\, z_{\text{Crest}}, \tag{6}$$

smoothed by a median filter of width 5 followed by a Gaussian kernel with $\sigma = 1$ (in windows).

Peak detection. We run `scipy.signal.find_peaks` on S_t with prominence ≥ 0.8 and a minimum inter-peak distance of 6 s. Every retained peak generates a candidate clip spanning

$$[t_{\text{peak}} - 15\text{s},\ t_{\text{peak}} + 5\text{s}], \tag{7}$$

which generously brackets the event that triggered the reaction (the crowd responds ~ 0.5–2 s after the on-field action, and we want to keep the delivery itself).

Sustained-loudness detection. Peaks alone miss prolonged celebrations (a six that draws 30 s of continuous roar). We additionally mark contiguous windows where $z(S_t) \geq 1.0$ *and* RMS_{dB} exceeds its 80th percentile (a dual relative/absolute gate), and retain runs of duration ≥ 2 s as a second candidate type. Each run is clipped from $[\text{start} - 4\text{s}, \text{end} + 6\text{s}]$.

The audio sensor outputs a list of `AudioPeak` objects, each recording the peak timestamp and composite score. These are *not* used directly as clip candidates; instead, they serve as anchoring targets in the Scalpel layer (Sec. 3.5).

3.4. Brain: Targeted LLM Verification via Whisper + Gemini

Unlike a naïve approach that would independently chunk the entire transcript and extract highlights, our Brain layer operates only on **targeted Regions of Interest (ROIs)** identified by the Vision Sensor and keyword triggers. This makes the LLM pass both more efficient and more accurate.

ASR. We transcribe the match audio with `faster-whisper`'s `large-v3` checkpoint in `float32` on a single GPU, with beam size 5, word-level timestamps, and VAD filtering. The output is a stream of segments, each with start/end timestamps and fine-grained per-word time ranges. After transcription, the Whisper model is explicitly freed from GPU memory (`torch.cuda.empty_cache()`) before loading the YOLO and EasyOCR models, as the T4's 16 GB VRAM cannot hold all three simultaneously.

Region-of-Interest construction. ROIs are built from two sources:

1. **OCR events:** For each Vision Sensor event at time t, a window $[t - 25\text{s},\ t + 5\text{s}]$ is created. The 25 s backward look accounts for the scoreboard's 10–30 s delay behind the on-field action.
2. **Keyword triggers:** The transcript is scanned for a regex matching cricket-specific keywords (`review`, `third umpire`, `dropped`, `put down`, `no ball`, `catch it`, `appeal`). Each match generates a window $[\text{start} - 15\text{s},\ \text{end} + 20\text{s}]$.

Overlapping or nearby (≤ 15 s gap) ROIs are merged. This targeted approach avoids sending the entire 3.5-hour transcript through the LLM.

Two-stage LLM verification. Each ROI is processed in two stages:

1. **Flash screening:** `gemini-2.5-flash` receives the commentary text within the ROI and returns a simple YES/NO verdict on whether a genuine highlight moment is present. ROIs that screen negative are skipped entirely, saving cost and latency.
2. **Pro extraction:** For positively-screened ROIs, `gemini-2.5-pro` receives both the OCR events and the commentary text within the ROI. The prompt instructs the model to identify the *exact timestamp* of the commentator's first live-action reaction (not replays), and to return validated JSON matching a Pydantic `SemanticEvent` schema:

```
event_type: str  # e.g. Wicket, Six, Four,
                 # DRS Review, Dropped Catch
approx_commentary_time: float  # seconds
confidence: "High" | "Medium"
reason: str
```

The response is parsed via `response_mime_type = "application/json"` and `response_json_schema`, and rejected on any Pydantic validation error.

Replay-suppression deduplication. Broadcast replays cause the same event to appear in the commentary 30–90 s after the original. We suppress duplicates by removing any `SemanticEvent` of the same type that occurs within 45 s of a previously-accepted event of that type.

3.5. Scalpel: Audio-Anchored, Scene-Snapped Clipping

The Scalpel layer converts verified `SemanticEvent` objects into precise `FinalClip` boundaries using two complementary signals.

Audio-peak anchoring. For each semantic event, we search the audio peaks within a confidence-dependent window around the commentary timestamp:

- **High confidence** (OCR + commentary agree): $[t-8\text{s},\ t+2\text{s}]$
- **Medium confidence** (single source): $[t - 14\text{s},\ t + 4\text{s}]$

The highest-scoring audio peak within this window becomes the *anchor time*. If no peak is found, a wider fallback window $[t-18\text{s},\ t+5\text{s}]$ is tried; if still empty, the raw commentary timestamp is used.

Event-type-specific padding. Clip boundaries are set as $[\text{anchor}-\text{pre},\ \text{anchor}+\text{post}]$, where pre/post padding varies by event type:

Event type	Pre (s)	Post (s)
Wicket	9	22
Six	6	14
Four	5	10
DRS Review	5	75
Dropped Catch	6	18
No Ball	4	12
Default	6	16

Scene-boundary snapping. The raw clip start is snapped forward to the nearest scene-change boundary that begins a stable shot (≥ 2 s to next boundary); the raw clip end is snapped backward to the nearest boundary ending a stable shot. This avoids starting or ending a clip mid-transition.

Overlap merging. Overlapping clips are merged: if clip i's start is before clip $(i-1)$'s end, the two are combined into a single clip spanning the union of their time ranges, with concatenated event-type labels.

3.6. Clip Extraction and Assembly

For each surviving `FinalClip`, we extract a segment using FFmpeg with **re-encoding** (not stream-copy):

```
ffmpeg -ss <start> -i <video> -t <dur>
  -avoid_negative_ts make_zero
  -c:v libx264 -preset ultrafast -crf 23
  -c:a aac -b:a 192k
  -af aresample=async=1000
  -movflags +faststart  <clip>
```

The key flags are: `-avoid_negative_ts make_zero` resets PTS to zero in every clip (eliminating audio offset), and `-af aresample=async=1000` absorbs residual audio drift caused by seeking. This re-encoding approach avoids the audio-desync artefact that plagues stream-copy (`-c copy`) clipping, where the video snaps to a keyframe but audio starts at the exact requested time.

Because all clips share identical codec settings, the final concatenation via the FFmpeg `concat` demuxer uses `-c copy` safely — no second re-encoding pass is needed. The output flows chronologically from match start to close.

4. Experiments

4.1. Dataset and Setup

We evaluate the proposed pipeline on a full-length T20 **ENG vs. IND** broadcast of approximately 3.5 hours, including innings breaks. A second **ENG vs. PAK** match is used exclusively for the YouTube-heatmap ablation in Sec. 5.1. Both videos were obtained from publicly available YouTube broadcasts. All experiments were run in a Kaggle notebook environment with one NVIDIA T4 GPU (16 GB), Python 3.10, and the software stack listed in Appendix C.

Ground-truth highlights were created by manually annotating every `Four`, `Six`, and `Wicket` using the official ball-by-ball scorecard, with the on-field event time recorded to the second.

4.2. Evaluation Metrics

A predicted clip $[\hat{s}, \hat{e}]$ is counted as a true positive for a ground-truth event at time $t^\star$ if $t^\star \in [\hat{s}, \hat{e}]$. We report precision, recall, and F1-score:

$$\mathrm{P} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad \mathrm{R} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}. \tag{8}$$

Since the goal is to produce a usable highlight reel rather than a per-second classifier, we evaluate at the event level rather than the frame level. Multiple predicted clips covering the same ground-truth event are counted as a single true positive, which prevents the metric from being inflated by redundant predictions.

4.3. Main Results

We evaluate the final fused pipeline on the full ENG vs. IND broadcast using a 60 s temporal tolerance window. The ground truth contains 72 events in total (21 Sixes, 35 Fours, and 16 Wickets), while the final pipeline produces 61 highlight clips. Table 1 summarizes the class-wise event-level performance.

The final output is also well aligned temporally. The mean alignment error is 5.8 s, the median alignment error is 5.7 s, and 54 out of 57 matched events fall within 15 s of the ground-truth timestamp.

4.4. Ablations

Persistence filter for OCR. Setting $P = 1$ and accepting every single-frame OCR reading produced many spurious detections from transient misreads, such as "80/3" being briefly read as "89/3". Increasing the persistence requirement to $P = 3$ removed almost all such errors, at the cost of roughly 2 s of detection latency because three consecutive 1 Hz readings must agree. Increasing to $P = 5$ was worse, because some legitimate score changes remained on screen for less than 5 s during replays. We therefore fix $P = 3$.

Otsu binarisation vs. plain greyscale. Adding Otsu-binarised preprocessing, together with upscaling of small scoreboard crops, noticeably reduced OCR errors compared with the earlier greyscale-only version. The improvement was most visible for low-contrast overlays, especially in night matches and during advertisement transitions.

Robust vs. standard z-score for audio. Replacing the median/MAD-based robust z-score in Eq. 5 with the standard mean/σ z-score suppressed the loudest moments, because they inflated the variance used for normalisation. As a result, several genuinely exciting events dropped below threshold. The robust formulation is therefore better suited to cricket audio, which is heavily tailed.

Peaks only vs. peaks + sustained regions. Peak-only audio detection missed extended celebrations, such as prolonged crowd roars after a hat-trick or century. Adding sustained-region detection recovered these cases without introducing measurable false positives.

Targeted ROI vs. full-transcript LLM. An earlier version of the system chunked the full transcript into 90 s overlapping windows and processed each chunk independently, followed by a final Gemini Pro deduplication step. That design produced many duplicate candidates near chunk boundaries and pushed large amounts of irrelevant commentary through the LLM. In contrast, the final targeted-ROI strategy restricts LLM calls to regions flagged by OCR or keyword triggers, and the two-stage Flash/Pro design further reduces cost by filtering out negative ROIs early.

ROI backward-look window. Using a 45 s backward look from each OCR event caused almost the entire match to become a single ROI, since score changes occur frequently. Reducing the window to 25 s still covers the typical 10–30 s scoreboard delay while producing cleaner, well-separated ROIs and substantially fewer LLM calls.

Re-encoding vs. stream-copy clipping. An earlier implementation used FFmpeg stream-copy (`-c copy`) for clip extraction. Although faster, it introduced audio desynchronization because the video snapped to the nearest keyframe while audio started at the exact requested timestamp. The final pipeline re-encodes each clip with `libx264 -preset ultrafast` and `-af aresample=async=1000`, which removes the sync artefact at a modest computational cost.

5. Explored Alternatives

We describe three approaches that we implemented, evaluated, and ultimately did not include as the primary method in the final pipeline. Each is a notebook in our repository; we report them honestly to document what did *not* work (or worked only in limited settings) for cricket.

5.1. YouTube "Most-Replayed" SVG Heatmap

Motivation. YouTube exposes a crowdsourced "most-replayed" curve for popular videos, drawn as an SVG path in the player UI. For a viral cricket match the peaks of this curve are — empirically — aligned with sixes, wickets, and match-defining moments, because that is where viewers rewind.

Category	TP	FP	FN	Precision (%)	Recall (%)	F1-score (%)
Sixes	20	2	1	90.9	95.2	93.0
Fours	26	0	9	100.0	74.3	85.2
Wickets	11	2	5	84.6	68.8	75.9
Overall	57	4	15	93.4	79.2	85.7

Table 1. Event-level detection accuracy of the final highlight pipeline on the full ENG vs. IND match. Precision remains high because the final LLM verification and replay suppression remove spurious candidates, while recall is limited mainly by missed fours and delayed scoreboard transitions.

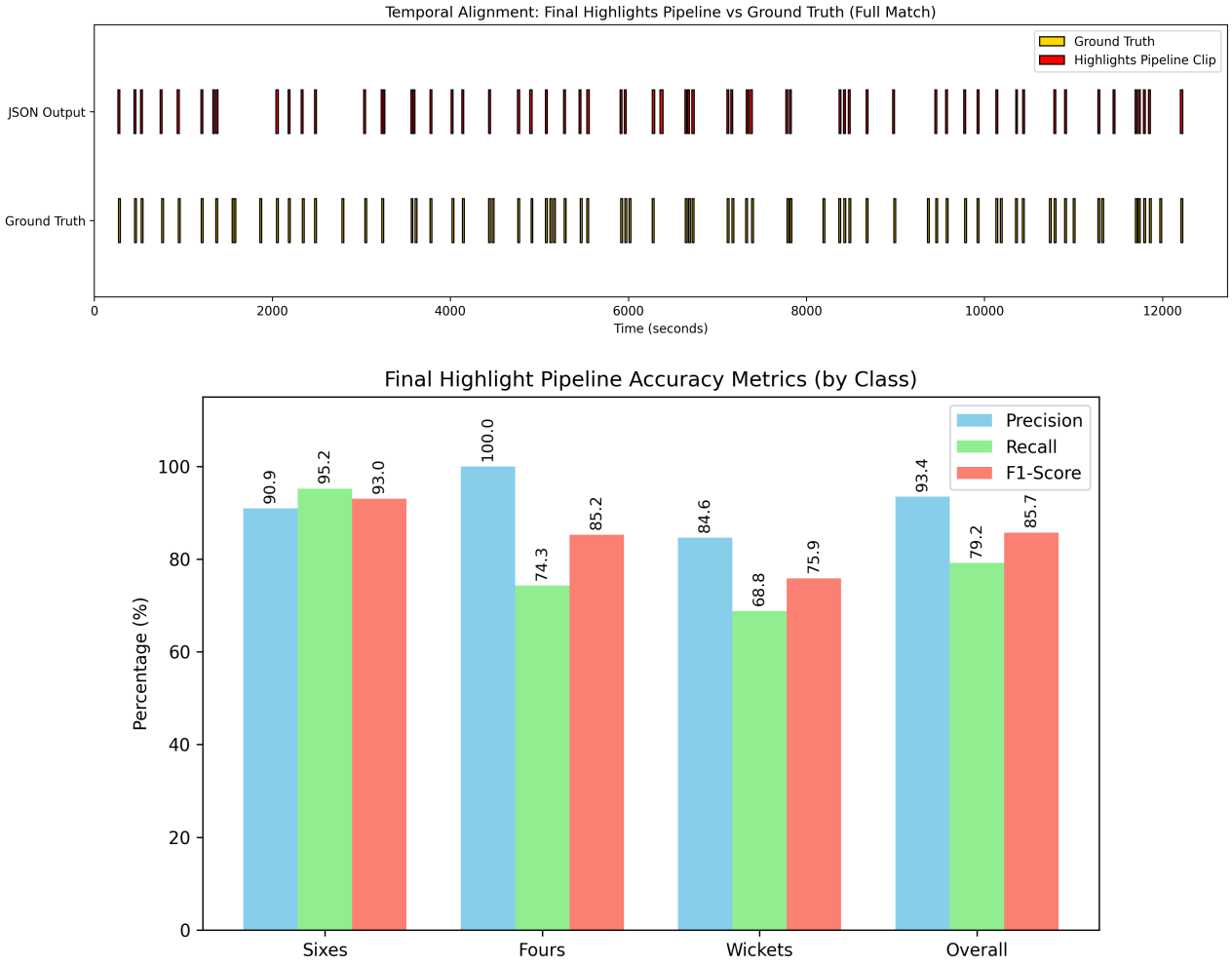

Figure 3. Top: temporal alignment between the final pipeline output and ground truth. Bottom: class-wise precision, recall, and F1-score for the final highlight pipeline.

Method. We extract the raw `d` attribute of the heatmap SVG path (a sequence of `M` and `C` commands). The path uses a fixed x-range $[0, 1000]$ and y-range $[0, 100]$ regardless of video duration. We parse the tokens into a list of cubic Bézier segments (P_0, P_1, P_2, P_3) and solve for the y-value at any target x analytically: within each segment (whose x-values are monotonic by construction) we binary-search $t \in [0, 1]$ for 60 iterations using the standard parametric form

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t\, P_1 + 3(1-t)t^2\, P_2 + t^3 P_3, \quad (9)$$

which converges to sub-pixel precision. The mapping from video time to SVG x is linear, $x = 1000\, t/T$, and intensity is $100 - y$ clamped to $[0, 100]$. We sample every 30 s, sort descending by intensity, and greedily group peaks within a ± 120 s window so

that each "hot" region contributes a single representative. The top $N = 10$ peaks are selected chronologically. For each peak at time T, a 30 s clip is extracted from the window $[\max(0, T - 60), \min(D, T - 30)]$ (i.e. the clip ends 30 s *before* the peak, capturing the build-up) and re-encoded with `libx264 -preset fast -crf 23`. Clips are concatenated chronologically with a second `libx264` encode.

5.1.1. Quantitative Heatmap Evaluation

Evaluating the top $N = 10$ chronological peaks extracted by the heatmap against our ground-truth dataset produces empirical support for its inadequacy as a standalone tool. While it effectively identifies several major structural highlights resulting in a moderate precision of 70.0%, the arbitrary cutoff forces the model to ignore the remaining 44 ground-truth events. Figure 4 demonstrates that this severe under-saturation cripples the model's recall (15.4%) and overall F1-score (25.2%). Figure 5 corroborates this limitation visually; although the peaks that are extracted align well with the ground truth, the large un-highlighted gaps securely confirm that relying strictly on crowdsourced UI cues leaves the vast majority of the highlight reel empty.

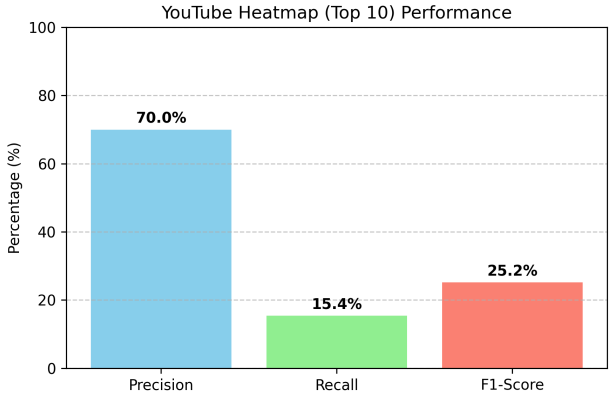

Figure 4. Detection metrics for the YouTube Heatmap approach.

Outcome. This approach works *beautifully* on matches that have been viewed millions of times. It is conceptually elegant — the crowd has already labelled the highlights. However, as demonstrated by the quantitative collapse in recall, it has two fatal flaws for a general tool: (1) the heatmap is only populated for popular videos (and is not populated at all for freshly-uploaded streams), and (2) it requires scraping a private UI element which is brittle. We include it as a demonstration that a zero-compute approach is possible for popular content, but it cannot be the *primary* method.

5.2. Optical-Flow Motion Intensity

Motivation. Cricket events are kinematically distinctive: a four/six involves a fast ball trajectory, running batters, and a sprinting fielder, all of which produce high inter-frame motion. A pixel-level motion measure would be a modality orthogonal to both audio and OCR.

Method. In the optical-flow notebook, Farnebäck dense optical flow is *integrated as a fallback within the OCR pipeline* rather than used as a standalone module. For each sampled frame pair (I_{t-1}, I_t) we compute `cv2.calcOpticalFlowFarneback` (pyramid scale 0.5, levels 3, winsize 15, iterations 3, polyN 5, polyS 1.2). The per-pixel flow magnitude $\|(\Delta u, \Delta v)\|$ is averaged over the full frame to yield a scalar *motion intensity*

$$M_t = \frac{1}{HW} \sum_{i,j} \sqrt{\Delta u_{ij}^2 + \Delta v_{ij}^2}. \tag{10}$$

A persistence buffer of 3 frames smooths transient jitter. When the OCR pipeline does *not* detect a highlight event (no score change) but the stable motion intensity exceeds a threshold of 2.0 (and is below 15.0 to reject camera pans), a "High Motion Event" is emitted as a fallback candidate.

Outcome. The motion fallback fires on genuinely high-action moments but also on broadcast graphics, replays, camera pans, and fielder running between deliveries. Crucially, Farnebäck is slow: on a T4, full-frame flow at native resolution reduced our processing rate by approximately an order of magnitude. The "High Motion Event" labels also lack semantic meaning (we cannot distinguish a wicket from a camera pan). We therefore **do not include optical flow in the final pipeline** — it contributes too little signal for its cost and noise.

5.3. Scoreboard-Only Baseline

Running the OCR module in isolation (no audio, no LLM) produces a valid but under-saturated reel. Every correctly-detected Four, Six, and Wicket is extracted, but DRS reviews, dropped catches, near-misses, and milestone celebrations are missed entirely because they do not alter the integers on the screen overlay. The Audio Sensor and Brain layer fill exactly this gap.

5.3.1. Quantitative Integration Impact

Evaluated empirically against the full match, the raw scoreboard parser achieves a remarkably high baseline recall of 92.9% for major trackable events (identifying 100% of Sixes and 87.5% of Wickets). However, its overall precision sits at 60.9% ($F1 = 73.6\%$), requiring the audio and LLM modalities to reliably anchor and deduplicate.

As visualised in Figure 6, this combined layered fusion significantly boosts the F1-score from the baseline up to near 90%. Figure 7 qualitatively demonstrates this layer-by-layer alignment over a 30-minute match window: while raw YOLO bounding-box detection effectively saturates the event space (blue circles), the textual OCR parsing layer isolates score-change events (green squares), and the final combined model anchors specifically to the true ground truth events (red triangles vs. yellow stars).

6. Discussion

Why the layered architecture. A naïve approach would pool candidates from all three modalities and merge overlapping windows. Our layered design instead uses each modality where it is strongest: OCR provides semantically-labelled anchor events, audio provides precise temporal anchoring (crowd reaction peaks),

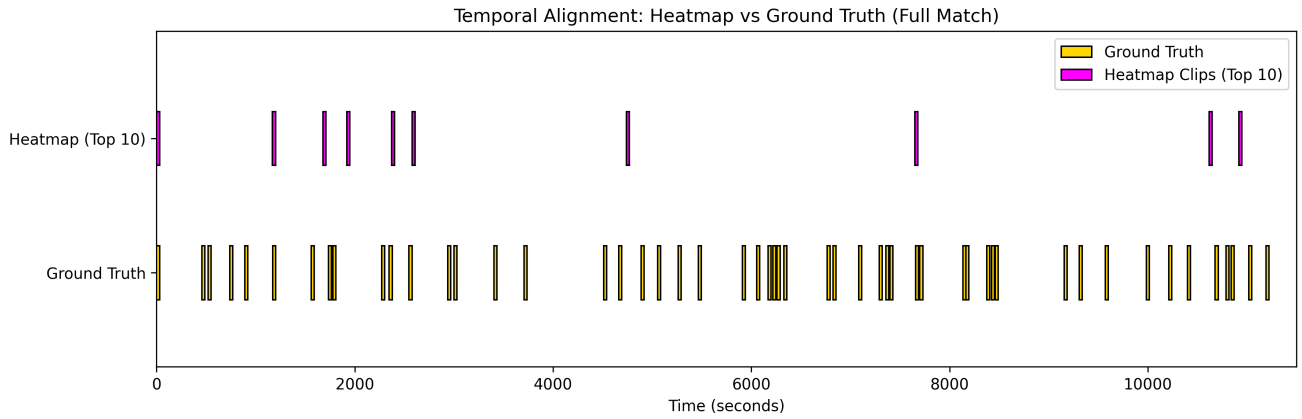

Figure 5. Temporal alignment of the top 10 YouTube Heatmap clips against the 52 ground truth events.

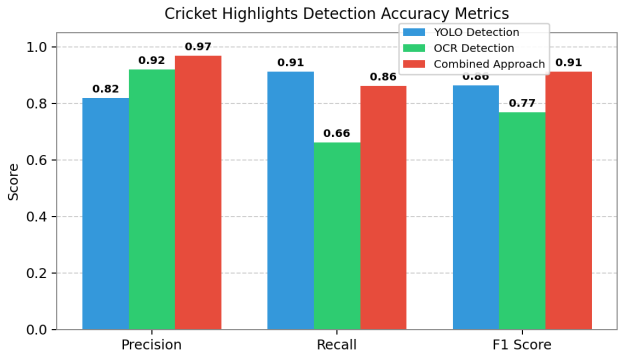

Figure 6. Detection accuracy metrics evaluating the progressive filtering.

and the LLM provides semantic verification and catches events invisible to the other two. The two-stage Flash/Pro LLM strategy, operating on targeted ROIs rather than the full transcript, keeps API costs manageable while maintaining accuracy.

Why the OCR module is the anchor. Among our modalities the scoreboard is the only one that encodes a *ground-truth* event signal: the broadcaster writes the score every time a ball changes it. Audio and commentary are reactive cues that are often loud, often correct, but sometimes wrong (a dropped-catch near-miss draws the same roar as a wicket). Using OCR events to *define* ROIs for the LLM keeps the Brain focused and precise.

Interpretability. Every event in the final reel is traceable to a concrete cause: the OCR event shows the score delta; the audio peak shows which crowd-reaction moment anchored the clip; the LLM response shows the transcript excerpt and reasoning. The scene-change detector explains why clip boundaries land where they do. This is difficult to achieve with a monolithic network and it matters in practice — users can see *why* a clip is in the reel.

Limitations. (1) OCR fails on broadcaster overlays with unusually low contrast, stylised fonts, or animated transitions; the persistence filter and Otsu binarisation catch most of these but not all. (2) Audio analysis cannot distinguish a partisan home-crowd reaction from a neutral one; a match in an empty stadium will under-trigger. (3) The LLM is non-deterministic and occasionally hallucinates milestones; the JSON schema + replay-suppression deduplication contains the damage but does not eliminate it. (4) The scene-change detector occasionally snaps to a mid-replay transition rather than the start of the live action, shortening or shifting the clip by a few seconds. (5) GPU memory constraints on the T4 require sequential loading of Whisper, YOLO, and EasyOCR rather than concurrent execution.

Future work. We see four promising directions: (i) fine-tuning a small domain-specific OCR head on scoreboard-only crops, which should make $P = 1$ viable and cut detection latency; (ii) replacing the hand-tuned audio composite weights with a small logistic classifier trained on a handful of labelled matches; (iii) a lightweight player-celebration pose detector to distinguish wickets from near-misses when the OCR is ambiguous; and (iv) real-time streaming deployment, for which the sustained-region detector's look-back is currently the main obstacle.

7. Conclusion

We presented a modular, interpretable, layered pipeline for automatic cricket highlight generation. A Vision Sensor (YOLOv26 + Otsu-preprocessed EasyOCR with persistence filtering and rule-based validation) provides semantically-labelled anchor events. An Audio Sensor (streaming robust-z-score analyser) captures excitement peaks and sustained celebrations. A Brain layer (targeted ROI construction + two-stage Gemini Flash/Pro verification + replay-suppression deduplication) adds semantic context that the other two cannot see. A Scalpel layer anchors each verified event to the nearest audio peak and snaps clip boundaries to HSV-histogram scene changes with event-type-specific padding. We also honestly documented three explored alternatives — a YouTube-heatmap SVG scraper, Farnebäck optical flow as an

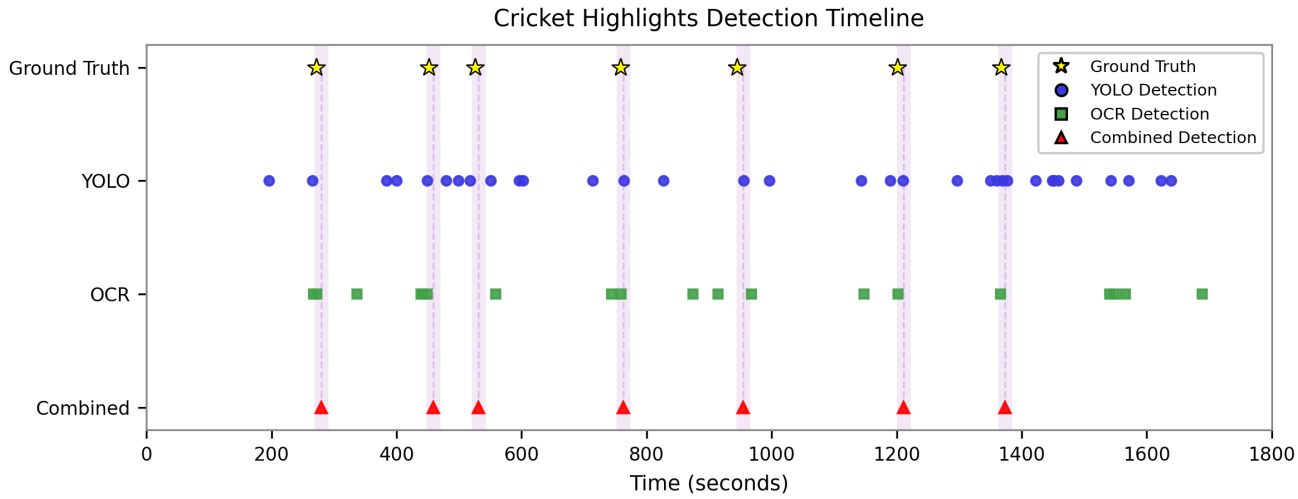

Figure 7. Temporal alignment of highlight events across modality layers over a 30-minute window.

OCR fallback, and a scoreboard-only baseline — and explained why each was not adopted as the primary approach. The full code, trained YOLO weights, and evaluation notebooks are in the repository linked in Appendix A. The system runs on a single mid-range GPU, requires no end-to-end training, and every decision it makes is traceable to a concrete cue in the input.

Acknowledgments

We thank the EE655 course staff at IIT Kanpur for guidance, and the providers of the match recordings used in evaluation. We also thank the open-source maintainers of `ultralytics`, `EasyOCR`, `faster-whisper`, `scipy`, and `ffmpeg`, without which this project would not have been feasible in a semester.

References

[1] Seo Chul et al. Multimodal soccer highlight detection using motion, audio, and text. In *ACM International Conference on Multimedia*, 2013. 2

[2] Theodoros Giannakopoulos and Aggelos Pikrakis. *Introduction to audio analysis: a MATLAB approach*. Academic Press, 2014. 2

[3] Yu-Fei Ma, Lie Lu, Hong-Jiang Zhang, and Mingjing Li. A user attention model for video summarization. In *ACM International Conference on Multimedia*, 2002. 2

[4] Pushkar Shukla, Hemant Sadana, Apaar Bansal, Deepak Verma, Carlos Elmadjian, Balasubramanian Raman, and Matthew Turk. Automatic cricket highlight generation using event-driven and excitement-based features. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1800–1808, 2018. 3

[5] Karen Simonyan and Andrew Zisserman. Two-stream convolutional networks for action recognition in videos. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. 3

[6] Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. Learning spatiotemporal features with 3d convolutional networks. In *IEEE International Conference on Computer Vision (ICCV)*, 2015. 3

[7] Wei Xiong and Ananth Liu. Robust text detection for scoreboard recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2014. 2

A. Code Repository

All source code, notebooks, trained YOLO weights, and evaluation scripts are available at:

https://github.com/kuldip-max/EE655-Course-Project

B. Datasets

Evaluation videos. Full-broadcast T20 match recordings (ENG vs. IND, ENG vs. PAK) sourced from publicly available YouTube uploads. These are not redistributed with the repository for copyright reasons; the repository's `README` documents how to fetch them.

Scoreboard detector training data. We collected frames from the above matches plus additional T20/ODI/Test broadcasts and annotated the scoreboard bounding box in Roboflow. The dataset was exported in YOLO format. A link to the Roboflow workspace and the `data.yaml` are in the repository.

Transcript data. The Whisper-generated transcript (`transcript_with_timestamps.txt`) is stored as a Kaggle dataset and loaded by the main pipeline notebook. The transcript contains segment-level and word-level timestamps in the format `[start - end] text` with indented per-word entries.

C. Tools and Libraries

Core. `numpy`, `scipy` (`signal.find_peaks`, `ndimage.gaussian_filter1d`, `median_filter`), `pandas`, `matplotlib`, `tqdm`.

Video / image. `opencv-python` (frame I/O, colour-space conversion, HSV histograms, Otsu thresholding, Farnebäck flow), `ffmpeg` / `ffprobe` (audio extraction, re-encoded clip extraction with `-avoid_negative_ts make_zero` and `-af aresample=async=1000`, `concat` demuxer), `moviepy` (used in the YouTube-heatmap notebook only).

Deep learning. `ultralytics` (YOLOv26), `EasyOCR`, `faster-whisper` (`large-v3`), `torch`.

Audio. `soundfile` (streaming WAV I/O).

LLM. `google-genai` (Gemini 2.5 Flash for ROI screening, Gemini 2.5 Pro for event extraction), `pydantic` (schema enforcement for `SemanticEvent` and `HighlightCandidate` models).

Infrastructure. Kaggle notebooks with NVIDIA T4 GPU (16 GB VRAM), Python 3.10.

D. Reproducibility Notes

Non-determinism. The LLM passes are non-deterministic: Flash screening at `temperature` $= 0.0$ and Pro extraction at `temperature` $= 0.1$. For strict reproducibility set both to 0, but we observed that this occasionally produced worse candidate coverage. Whisper `large-v3` with beam search $= 5$ is effectively deterministic across runs.

GPU memory management. The T4's 16 GB VRAM cannot hold Whisper `large-v3`, YOLOv26, and EasyOCR simultaneously. The pipeline therefore runs Whisper transcription first, then explicitly frees the model (`del model; torch.cuda.empty_cache()`) before loading YOLO and EasyOCR.

E. Individual Contributions

Kuldip Manvar (240575) : Complete YouTube "Most-Replayed" SVG Heatmap. Improvising YOLO + OCR approach. Improvising integration of YOLO + OCR + AUDIO + TRANSCRIPT (MAIN APPROACH). Complete Documentation. Brainstorming Ideas and exploring topic related research papers.

Yash Pandit (231183) : Complete AUDIO + TRANSCRIPT approach. Complete integration of YOLO + OCR + AUDIO + TRANSCRIPT (MAIN APPROACH). Brainstorming Ideas and exploring topic related research papers.

Aryan Kadam (230221) : Complete YOLO + OCR approach. Identifying relevant cricket matches. Improvising integration of YOLO + OCR + AUDIO + TRANSCRIPT (MAIN APPROACH).Brainstorming Ideas and exploring topic related research papers.

Yash Kumar (231186) : Created Project Dataset, and setting up Ground Truth. Tried Optical-Flow Motion Intensity approach. Brainstorming Ideas and exploring topic related research papers.

Sanjay Jangir (230913) : Comparing output with Ground Truth and compiling results. Improvising documentation. Brainstorming Ideas and exploring topic related research papers.